Universidad Europea

Sistemas de percepción UA5

Sistema de prevención de colisión en robots autónomos en fábricas usando sistemas visuales

Miembros de grupo: Juan David Velasquez Bran, Gerardo Antonio Ceballos van Grieken, Toni van Ginkel

Introducción

Este proyecto implementa un sistema de percepción visual para la prevención de colisiones en robots autónomos dentro de un entorno de fábrica. El objetivo es que un robot móvil, como los AGV que circulan por los pasillos de un almacén, pueda detectar objetos en su camino, estimar cómo se mueven y decidir si debe frenar, ceder el paso o continuar.

Para conseguirlo se ha optado por un enfoque híbrido: una red neuronal pre entrenada (YOLO) detecta y clasifica los objetos en cada fotograma, y su propio tracker (ByteTrack) les asigna un ID estable entre fotogramas. A partir de ese seguimiento, un conjunto de técnicas clásicas calcula tanto el movimiento lateral de cada objeto, su velocidad y dirección en la imagen, como su movimiento en profundidad, si se acerca o se aleja de la cámara, llegando a estimar el tiempo que tardaría en entrar en contacto. Esta combinación permite aprovechar la capacidad de detección del aprendizaje profundo sin necesidad de entrenar ni etiquetar un dataset propio, manteniendo bajo control la parte de cálculo cinemático con técnicas clásicas vistas en la asignatura.

Descripción del escenario robótico

El escenario elegido es una fábrica o almacén en el que uno o varios robots autónomos (AGV) se desplazan por pasillos compartidos con personas, cajas y otros vehículos. El riesgo principal en este tipo de entornos es la colisión con obstáculos que entran en el camino del robot de forma imprevista, por lo que el robot necesita un sistema de percepción capaz de detectar esos objetos con suficiente antelación.

Justificación del enfoque

Se ha descartado un enfoque puramente clásico porque reconocer y clasificar objetos variados en condiciones de iluminación e indeterminación de la escena es un problema donde las redes de deep learning superan claramente a métodos basados en descriptores (ORB, SIFT) o en segmentación por umbral. No tenía sentido entrenar una red desde cero ni montar un dataset propio para esta práctica, así que se ha optado por transfer learning: usar YOLO con los pesos ya entrenados sobre el dataset COCO, sin reentrenamiento.

La parte de seguimiento entre fotogramas (saber qué objeto detectado ahora es el mismo que antes, y calcular cómo se mueve) sigue resuelta con un algoritmo clásico, no con otra

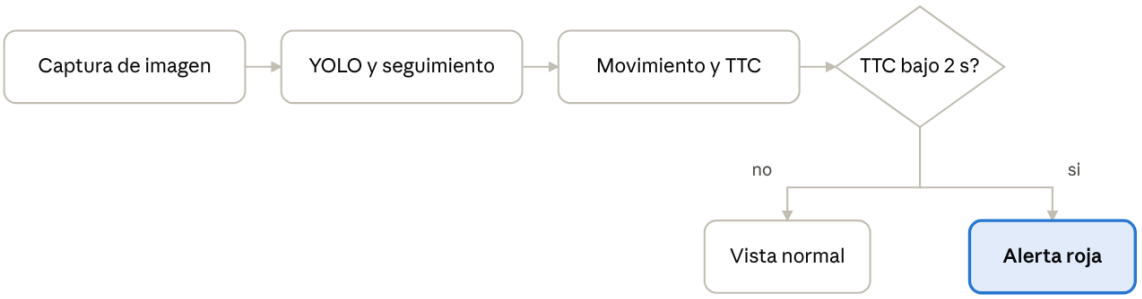
red neuronal. En la versión final se usa el tracker integrado de YOLO (ByteTrack, u opcionalmente BoT-SORT), que asigna un ID estable a cada objeto combinando posición y apariencia, en lugar del seguimiento por centroides más simple de las primeras pruebas que habíamos hecho. Sobre ese historial de posiciones se aplica otra técnica clásica, puramente geométrica: un ajuste por mínimos cuadrados que permite estimar no solo el movimiento lateral, sino también si el objeto se acerca o se aleja de la cámara a partir de cómo crece o encoge su caja, sin necesidad de calibrar la cámara ni conocer el tamaño real del objeto.

Descripción del sistema y flujo de procesamiento

El sistema procesa cada fotograma, ya sea en directo desde una cámara o desde un archivo de video, siguiendo estos pasos:

1. Captura del fotograma, desde la cámara o desde el archivo de video.
2. YOLO detecta y clasifica los objetos, y su propio tracker (ByteTrack por defecto, o BoT-SORT) les asigna un ID estable entre fotogramas.
3. Para cada ID se guarda un historial reciente (los últimos 0.5 segundos aproximadamente) de la posición y el tamaño de su caja.
4. Con ese historial se ajustan rectas por mínimos cuadrados para estimar dos tipos de movimiento: el lateral, en el plano de la imagen (velocidad y dirección), y el de profundidad, es decir, si el objeto se acerca o se aleja de la cámara, a partir de cuánto crece o encoge su caja por segundo.
5. Cuando un objeto se acerca, se calcula el tiempo hasta el contacto (TTC), como 1 dividido entre el ritmo de crecimiento de su caja, sin necesidad de conocer la distancia real ni el tamaño del objeto.
6. Se aplican umbrales con histéresis para decidir si un objeto está quieto o en movimiento, y por defecto solo se muestran en pantalla los que están en movimiento, para no saturar la imagen con objetos estáticos del fondo.
7. Si el tiempo hasta el contacto baja de un umbral de alerta, el cuadro del objeto se resalta en rojo con un aspa, como aviso visual de posible colisión.

El sistema avisa cuando el tiempo hasta el contacto baja de 2 segundos



Dataset y metodología de validación

Al usar transfer learning con YOLO, no se ha entrenado ni se ha necesitado un dataset etiquetado propio. Los modelos de detección de YOLO26 se distribuyen preentrenados sobre el conjunto COCO, que abarca 80 clases de objetos. COCO contiene unas 118 000 imágenes de entrenamiento y 5 000 de validación de escenas cotidianas, con los objetos anotados mediante cajas delimitadoras. Las 80 clases cubren personas, vehículos (bicicleta, coche, moto, autobús, camión, tren, avión, barco), mobiliario urbano (semáforo, señal de stop, banco), animales domésticos y salvajes, accesorios personales (mochila, paraguas, maleta), material deportivo, menaje y alimentos, muebles, electrodomésticos y dispositivos electrónicos. Al no haberse reentrenado el modelo, el sistema solo puede reconocer y seguir objetos de estas categorías. Cualquier otro objeto que se mueva delante de la cámara no se detecta, o se confunde con la clase de COCO más parecida.YOLO tiene varios modelos entrenados con distintas capacidades. Nosotros usamos el yolo26n.pt, nano, el más liviano. En diversas pruebas que hicimos probando modelos de mayor capacidad la velocidad de procesamiento en tiempo real descendía a niveles que no nos parecieron aceptables.

La solución se probó de varias maneras:

- Con un video grabado en primera persona dentro de un almacén real.
- Con captura en línea desde una cámara web con acercamientos de la parte trasera de un automóvil y de un celular.

En la evaluación ONLINE se capturó el video a resolución de 640x480 píxeles y 30 FPS. Con estas condiciones los FPS reales de procesamiento se ubicaban alrededor de 22 y cuando probamos la cámara a resolución 1280x960 obtuvimos 5 FPS pero esta limitación estaba causada por las limitaciones de hardware en el manejo de la captura que del procesamiento.

Los valores de velocidad reportados por defecto están en unidades de píxeles por segundo. Si quisiéramos obtener medidas en cm/s ó m/s, por ejemplo, tendríamos que estimar la distancia de cada objeto a la cámara y/o usar algún tipo de calibración que nos permita transformar de píxeles a medidas de distancia real.

Implementación

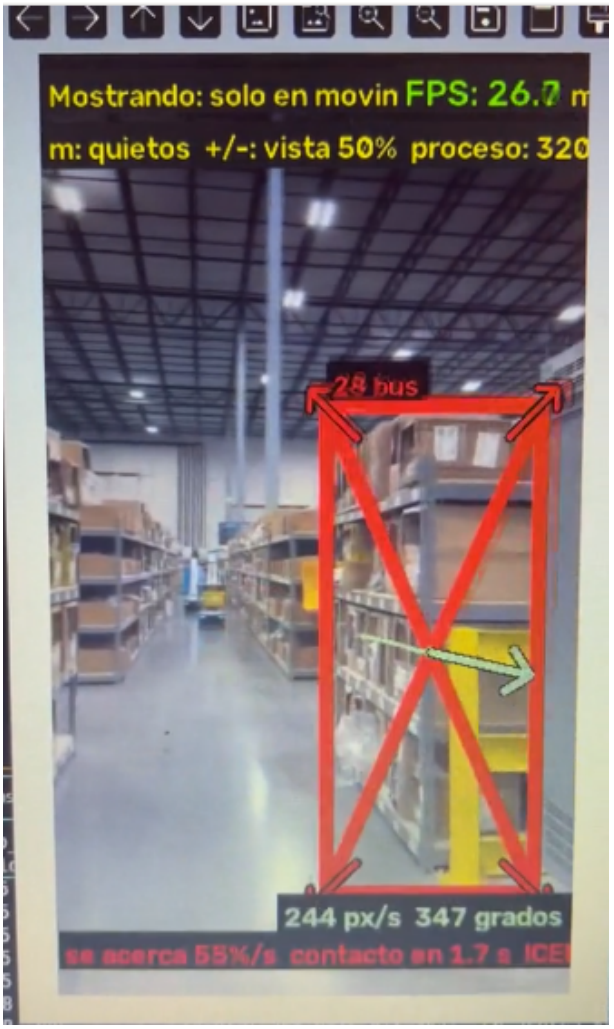
El sistema está implementado en Python, usando la librería YOLO de Ultralytics. Se carga YOLO y se ejecuta la detección y el seguimiento en un solo paso (método track, con ByteTrack o BoT-SORT). OpenCV solo se usó para manejar los objetos gráficos y anotaciones que se hacen en cada frame del video resultado. La librería lap es necesaria para que el tracker pueda asociar las detecciones entre fotogramas.

El trabajo de ajuste se ha centrado en los parámetros del sistema: el umbral mínimo de confianza, la resolución de proceso que usa YOLO (que puede reducirse para ganar velocidad a costa de detectar peor los objetos pequeños o lejanos), la ventana de tiempo e historial mínimo con la que se calcula el movimiento, los umbrales con histéresis que

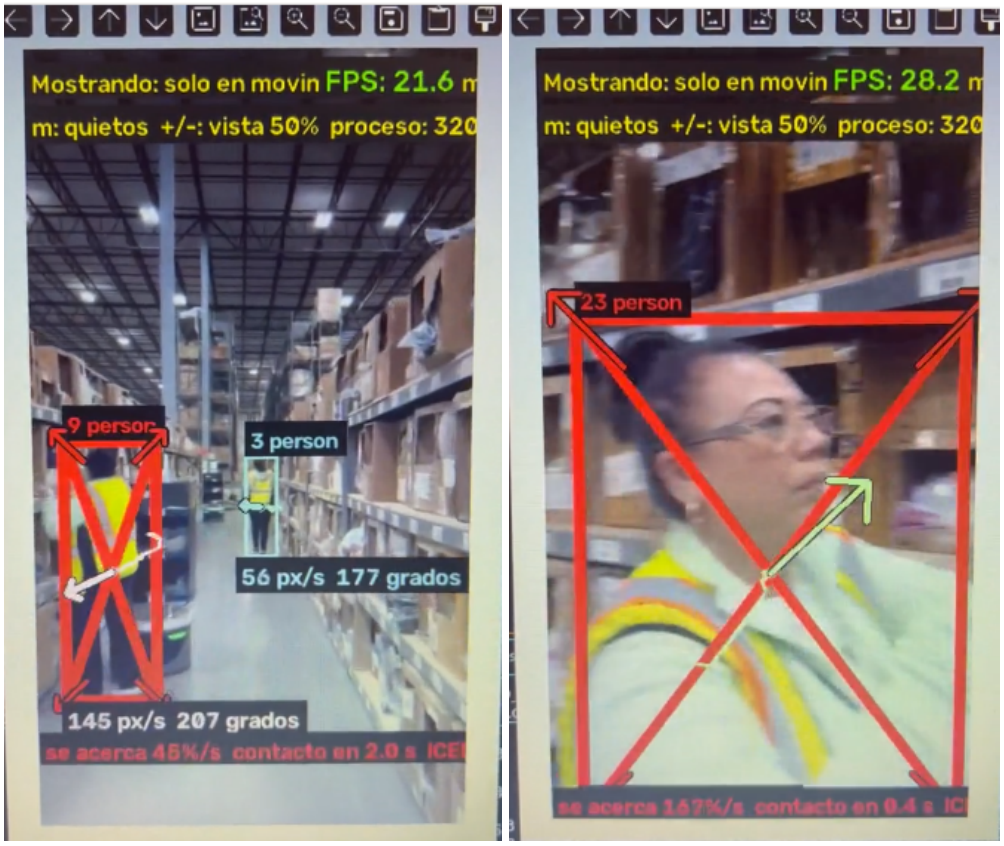
separan un objeto quieto de uno en movimiento (tanto en el plano de la imagen como en profundidad), el tiempo que se tolera sin ver un objeto antes de olvidarlo, y el umbral de tiempo hasta el contacto que dispara la alerta visual. La velocidad lateral se puede mostrar en metros por segundo si se indica manualmente cuántos píxeles equivalen a un metro a la distancia del objeto, y en píxeles por segundo si no se indica.

Resultados: casos de acierto y error

La siguiente figura muestra un caso de error en el que se detecta un objeto clase Bus en los racks del almacén. Es de notar que como lo importante es que el robot obtenga una señal de alerta ante una posible colisión con un objeto, la detección de una clase correcta no es tan importante en este caso.



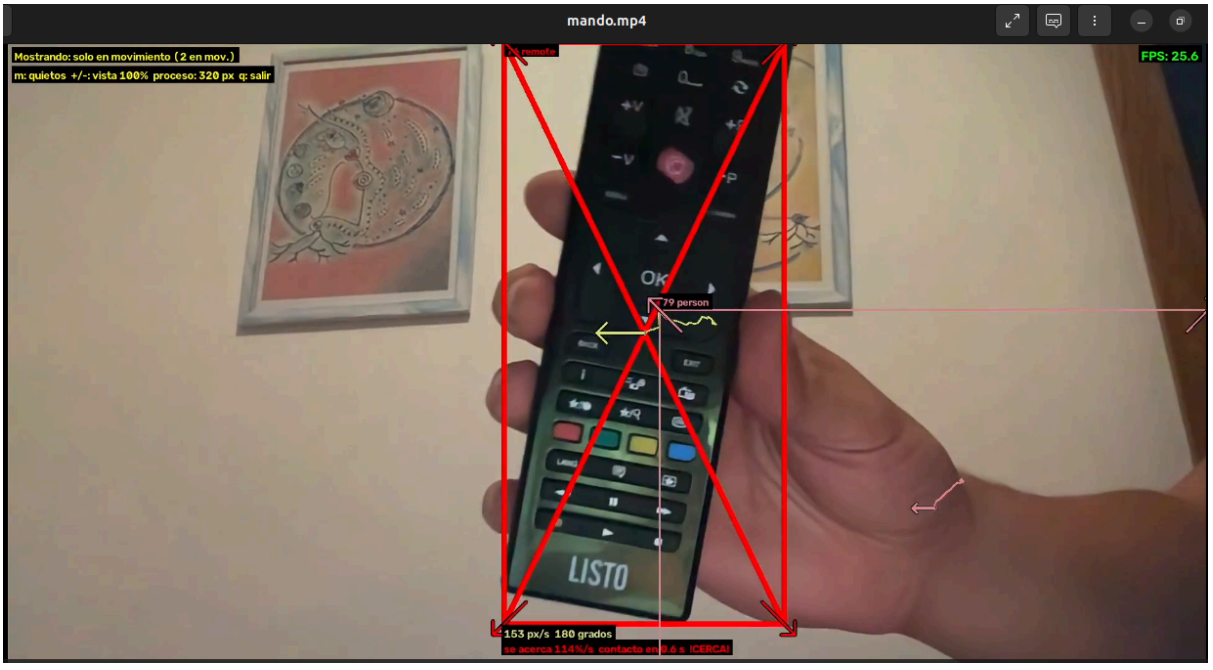
Igualmente mostramos a continuación casos de acierto en la detección de dos personas, objetos posibles a colisionar en el video de prueba.



A continuación una captura del vídeo resultante en una prueba ONLINE en la que nos acercamos rápidamente a un auto estacionado y se detectó correctamente el posible objeto de colisión.



El video completo se puede obtener en el repositorio GitHub.
A continuación otra prueba ONLINE acercando rápidamente un Mando de Control.



Conexión entre la percepción visual y la acción robótica

A diferencia de las primeras versiones del sistema, en las que esta conexión era solo una propuesta de diseño, en la versión final ya está implementada una primera forma de ella: cuando un objeto se está acercando a la cámara y su tiempo estimado hasta el contacto

cae por debajo de un umbral (2 segundos por defecto), el sistema lo marca con una alerta visual clara, un cuadro rojo grueso con un aspa encima del objeto.

Esa alerta es exactamente la señal que un robot real usaría para decidir una acción, por ejemplo frenar, detenerse o esquivar, antes de que la colisión sea inevitable. En esta práctica la acción robótica en sí no se ha llegado a ejecutar sobre un robot físico, pero la detección del momento crítico, que es la parte difícil del problema, sí queda resuelta y visualizada en el video de demo.

Lo que pasaría en caso de que un robot detectase una colisión con un objeto que se está moviendo, sería frenar para evitar dicha colisión. En caso de que frenando no sea suficiente para evitar la colisión (como por ejemplo en el caso que el objeto se esté acercando a la posición de nuestro robot) habrían varias opciones que el robot podría tomar:

- El robot podría generar avisos audiovisuales para alertar al objeto que se está acercando, asumiendo que dicho objeto sea controlado por un humano.
- Otra opción sería, si las circunstancias lo permiten, que el robot se mueva a un lado que permita evitar dicha colisión, pero esto requeriría más sensores y cámaras en los laterales para detectar que hay espacio libre y que eso no causaría otra colisión

Limitaciones y líneas de mejora futuras

El sistema tiene varias limitaciones que conviene reconocer. La primera sigue siendo el rendimiento: YOLO ejecutándose en tiempo real tiene un coste que depende del hardware disponible, y aunque la versión final permite reducir la resolución de proceso para ganar velocidad, eso se hace a costa de detectar peor los objetos pequeños o lejanos.

La segunda es que la velocidad lateral solo se puede mostrar en unidades reales (metros por segundo) si se indica manualmente cuántos píxeles equivalen a un metro a la distancia del objeto, un valor que hay que calibrar a mano para cada escena y que no se adapta solo si el objeto cambia de distancia. Sin ese dato, la velocidad queda en píxeles por segundo.

La tercera es que sigue siendo un método clásico que puede confundir identidades en cruces muy cercanos entre objetos. La estimación de profundidad, por su parte, asume que el tamaño aparente del objeto cambia de forma más o menos constante durante la ventana de medida, así que una oclusión parcial o un giro del objeto puede distorsionar el ritmo de crecimiento medido y dar un tiempo hasta el contacto poco fiable. Una de las principales fuentes de error en el cálculo del porcentaje de acercamiento, que es el parámetro importante para estimar el tiempo de colisión, es la gran variabilidad en el tamaño de las cajas que produce el tracker automático de YOLO para un mismo objeto.

Como líneas de mejora futura se podría sustituir el factor de escala manual por una calibración automática de la cámara, afinar los umbrales de histéresis con pruebas reales en vez de valores fijados a mano, y probar con más de una cámara para estimar la distancia real sin depender de la variación de tamaño de la caja.

Una grán mejora en la estimación del aumento de tamaño del objeto debido al acercamiento de la cámara se pudiera lograr explorando algoritmos de detección de *landmarks* dentro de

las cajas. Luego se haría seguimiento a los landmarks y se estimaría la transformación (escala y rotación) entre distintos frames. Esto solventaría en gran medida los errores causados por la variabilidad en el tamaño de las cajas que arroja YOLO.

Conclusiones

Este proyecto demuestra que es posible construir un sistema de percepción útil para la prevención de colisiones combinando una red de detección preentrenada con técnicas clásicas de seguimiento, sin necesidad de entrenar un modelo desde cero. El resultado no es un sistema listo para producción, pero sí una prueba de concepto que conecta de forma clara la percepción visual con una posible decisión robótica, y que deja identificadas sus limitaciones principales (rendimiento, falta de calibración, robustez del seguimiento, precisión en la estimación del porcentaje de acercamiento) como punto de partida para un desarrollo futuro.